

第4章

中断系统与GPIO



第4章 中断系统与GPIO



- 4.1 中断系统
- 4.2 数字I/O端口
- 4.3 端口应用



4.1 中断系统

- 中断系统是现代微处理器的一个重要组成部分。
- 在MSP430单片机中其作用更加突出，它不但具有传统中断系统的作用，而且还被赋予了新的角色。
- 充分而有效地利用MSP430单片机的中断可以简化程序和提高执行效率。

4.1 中断系统



- 4.1.1 中断系统基本概念
- 4.1.2 中断类型
- 4.1.3 中断响应过程
- 4.1.4 中断函数
- 4.1.5 鲲鹏920的中断系统

4.1.1 中断系统基本概念



1. 中断与中断系统

● 中断

CPU暂时中止正在执行的程序，转去执行其他工作(中断服务程序)，处理完毕后又回到被中止的程序处继续执行的过程。

4.1.1 中断系统基本概念



● 中断源

- ◆ 把引起中断的事件，或者能够发出中断请求信号的来源、电路。
- ◆ 早期的中断源主要来自外部硬件，即硬件中断。
- ◆ 由于中断在提高CPU利用率方面效果十分明显，目前中断已不再局限于硬件中断，单片机内部模块也可以触发中断。

4.1.1 中断系统基本概念



● 中断系统

- ◆ 是指实现中断功能的软硬件系统。
- ◆ 中断系统能够在 CPU 转去执行中断程序前**自动保护中断现场**，并在中断程序执行完返回时能够**自动恢复中断现场**，以准确返回到断点处继续执行被中断的程序。
- ◆ 中断现场的保护与恢复通常由堆栈来完成。

4.1.1 中断系统基本概念



- 中断系统的功能：

- ◆ 正确识别中断请求，实现中断响应，中断处理及中断返回
- ◆ 实现中断优先级排队
- ◆ 实现中断嵌套

4.1.1 中断系统基本概念



2. 中断向量与中断标志位

● 中断向量

- ◆ 中断向量用于存放中断服务程序的入口地址。
- ◆ 一旦中断源的中断请求被CPU响应，CPU就会根据中断源的中断向量准确地找到中断服务程序的入口地址，进而执行相应的处理程序。
- ◆ 通常把中断向量按照一定规律集中存放在存储器的特定区域。该区域即被称为中断向量表。



4.1.1 中断系统基本概念

● 中断向量

- ◆ 在MSP430单片机中，中断向量表以表格的形式存放在最高地址为0x0FFFE的一个连续区域内，其组织形式如表所示。
- ◆ 表中每一行存放一个中断向量，表中的行号称为**中断类型号或中断号**。
- ◆ 中断号(行号)与中断向量之间建立起一一对应的关系，即中断号为 n 的中断向量为 $0xFFFE-2n$ 。
- ◆ 中断向量的长度是一个字长，即两个字节，并且是偶地址对齐的。

中断向量表样式

0x0FFFE	中断向量 0
0x0FFFC	中断向量 1
...	...
0xFFFE-2n	中断向量 n

4.1.1 中断系统基本概念



● 中断标志位

- ◆ 当有多个中断源同时向CPU发出中断请求时，CPU会根据这些中断的优先级依次进行响应。
- ◆ 为了不使那些未被立即响应的中断请求丢失，必须将其保存起来以便响应高优先级中断请求后，再接着响应这些优先级低的中断请求。
- ◆ 通常用一个二进制位来标识中断源是否产生过中断请求，1表示中断源已经产生过中断请求，0表示中断源未发生过中断请求。这些二进制位被称为**中断标志位**。
- ◆ 中断标志位与中断源是一一对应的，利用相应的中断标志位就可以确定产生中断请求的中断源。
- ◆ 用于存放中断标志位的寄存器被称为**中断标志寄存器**。

4.1.1 中断系统基本概念



3.中断优先级与中断嵌套

● 中断优先级

- ◆ 为使系统能及时响应并处理发生的所有中断，MSP430单片机系统根据触发中断事件的重要性和紧迫程度，将中断源划分为若干个级别，称作中断优先级。

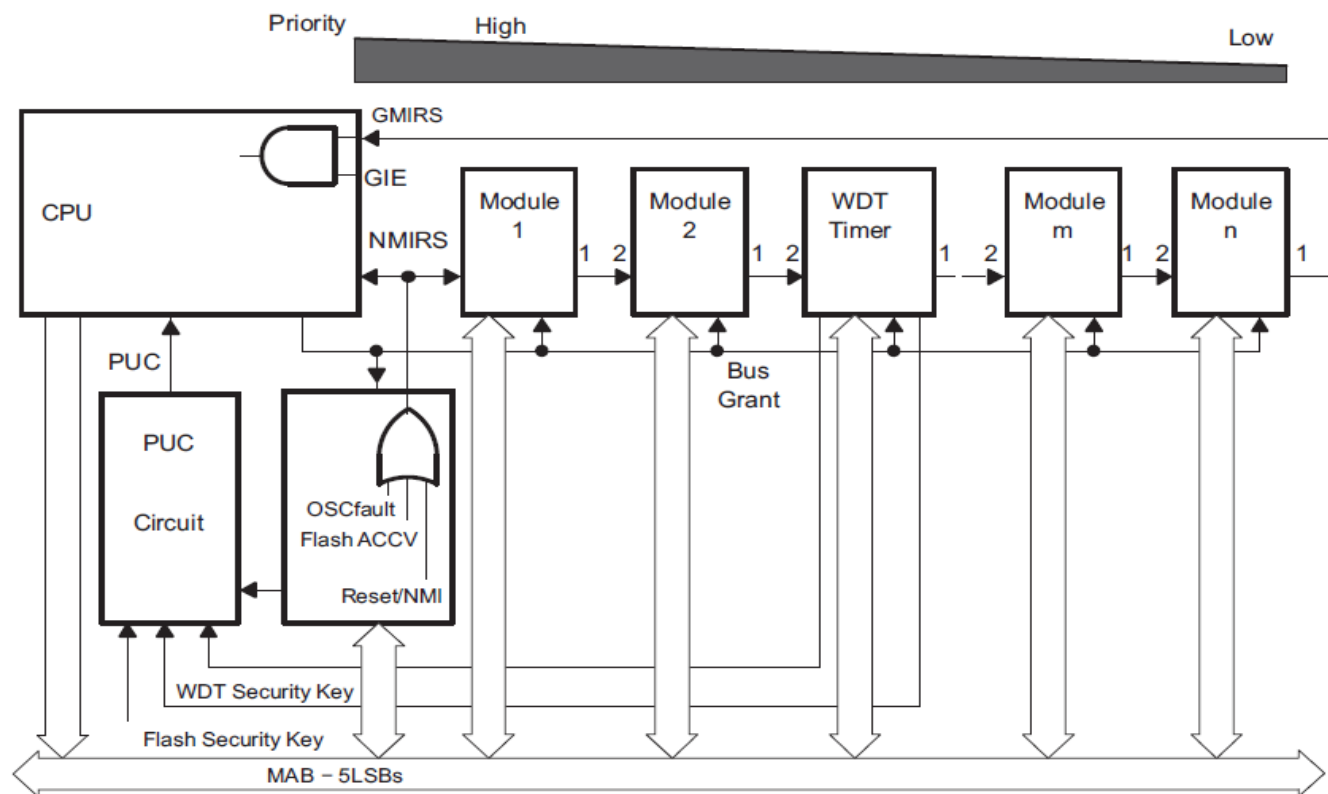
4.1.1 中断系统基本概念



● 中断优先级

MSP430的中断优先级仲裁方式使用的是链式结构，优先级固定。优先级别取决于该模块在链中的位置，越靠近CPU/NMIRS，级别越高。

中断裁决电路会根据已确定的优先级进行裁决：级别高的先得到响应。



There are three types of interrupts:

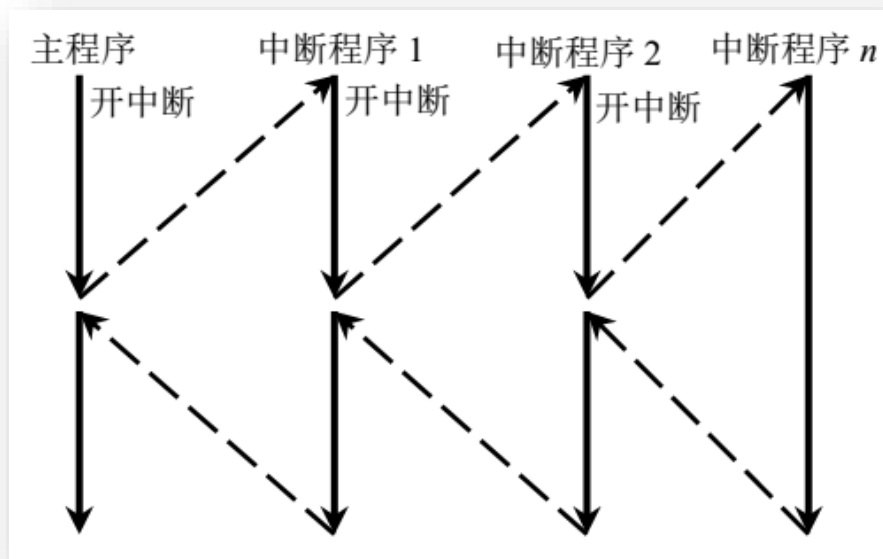
- System reset
- (Non)-maskable NMI非屏蔽中断
- Maskable可屏蔽中断

4.1.1 中断系统基本概念



● 中断嵌套

指正在执行中断服务程序时，被更高优先级的中断请求打断，转而执行更高优先级的中断服务程序，待处理完毕后再返回到被中断了的中断服务程序后继续执行的过程。





4.1.1 中断系统基本概念

- ◆ 由中断优先级可知，当多个中断同时向CPU提出中断请求时，中断系统会选择一个优先级较高的中断请求进行响应。
- ◆ 在CPU响应中断请求的时候，会将状态寄存器清零，即关闭了总中断使能位。

状态寄存器 (SR) 各位的定义：

15-9	8	7	6	5	4	3	2	1	0
保留	V	SCG1	SCG0	OSCOFF	CPUOFF	GIE	N	Z	C

- ◆ 在此情况下，不论优先级高低，任何可屏蔽中断都无法得到响应。因此，默认情况下，MSP430无法实现中断嵌套，要实现需满足一定条件。

4.1 中断系统



- 4.1.1 中断系统基本概念
- 4.1.2 中断类型
- 4.1.3 中断响应过程
- 4.1.4 中断函数
- 4.1.5 鲲鹏920的中断系统

4.1.2 中断类型



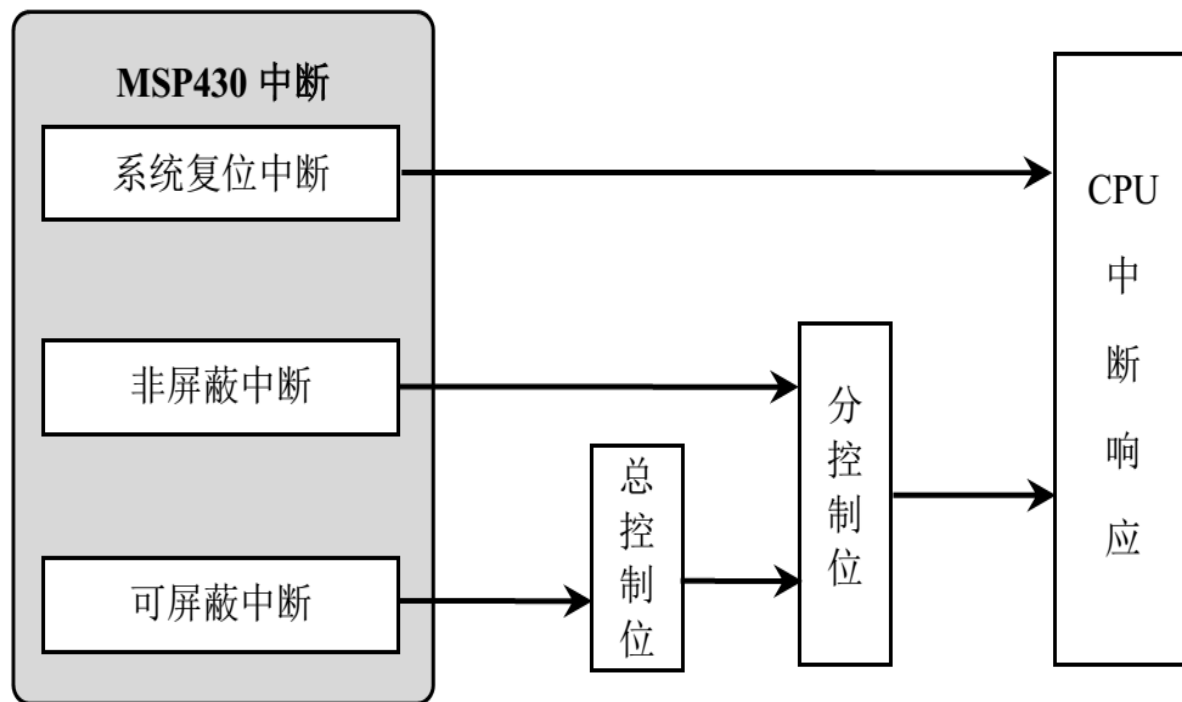
根据触发中断的信号是来自单片机内部还是来自外部引脚可分为**外部中断**与**内部中断**。

- ◆ 由外部引脚信号触发的中断称为**外中断**，如通过P1.0~P1.7、P2.0~P2.7等引脚触发的中断。
- ◆ 由单片机内部模块触发的中断称为**内中断**，如定时器中断、串行通信接收与发送中断等。

4.1.2 中断类型



根据中断源的可控程度可将MSP430单片机中断分为系统复位中断、非屏蔽中断与可屏蔽中断。

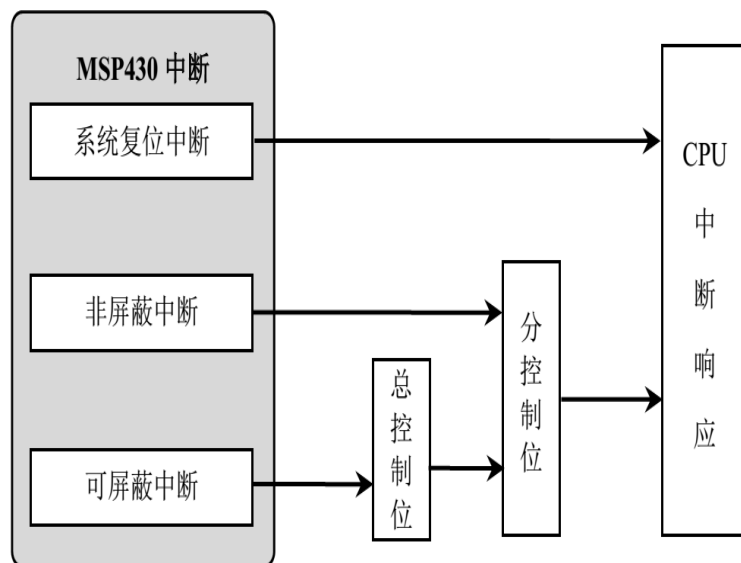


4.1.2 中断类型



1. 系统复位中断

- 系统复位(system reset)中断由触发系统复位中断的中断源产生，它既不受控于总中断使能位，也不受控于自身的使能位。
- 因此，它的**中断优先级最高**，只要有中断请求就会优先得到响应。



4.1.2 中断类型



1. 系统复位中断

● 能够触发系统复位中断的中断源有：

◆ (1) 掉电复位；

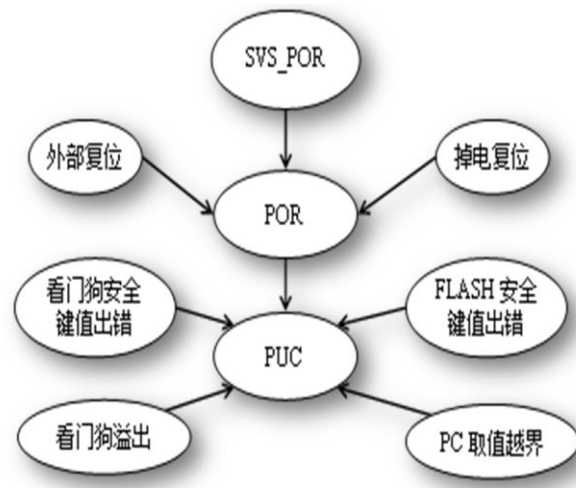
◆ (2) 复位模式下，当Reset引脚为低电平时；

◆ (3) 看门狗模式下，看门狗定时溢出；

◆ (4) 看门狗安全键值出错；

◆ (5) FLASH安全键值出错；

◆ (6) PC值超出范围。



4.1.2 中断类型



1. 系统复位中断

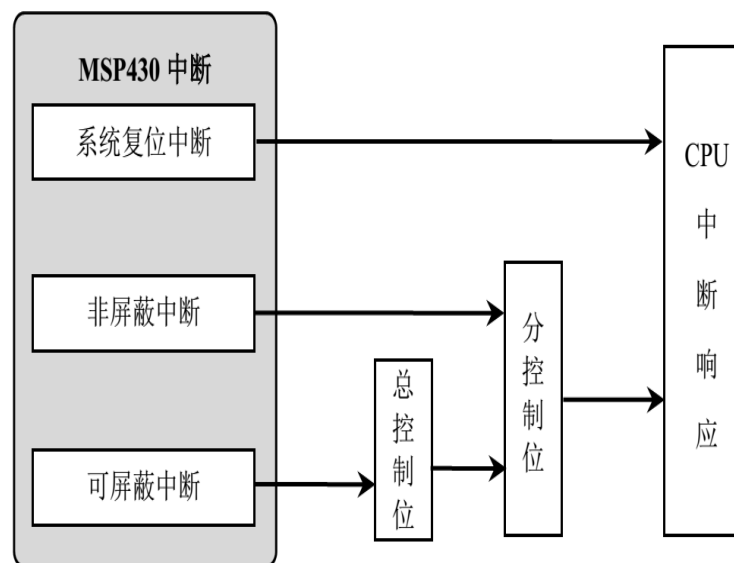
- 观察这些中断源不难发现，以上6种情况均是引起系统复位的原因。所以说只要有系统复位信号(POR&PUC)产生，就会触发系统复位中断。
- 尽管这么多中断源均可触发系统复位中断，但是在MSP430单片机中系统复位中断的入口地址只有一个，即0xFFFFE。因此，系统复位中断是多源中断。

4.1.2 中断类型



2. 非屏蔽中断

- 非屏蔽中断不能被总中断使能位控制，但可以被各自的中断使能位(分控制位)控制
- 与系统复位中断一样，非屏蔽中断也只有一个固定的中断入口地址(0x0FFFC)。



4.1.2 中断类型



2. 非屏蔽中断

- 在MSP430F261x中能够触发非屏蔽中断的中断源只有三种，分别是

- ◆ 外部中断或非屏蔽中断NMI

- ◆ 振荡器失效：未起振、突然停止工作、振荡器关闭

- ◆ FLASH访问出错

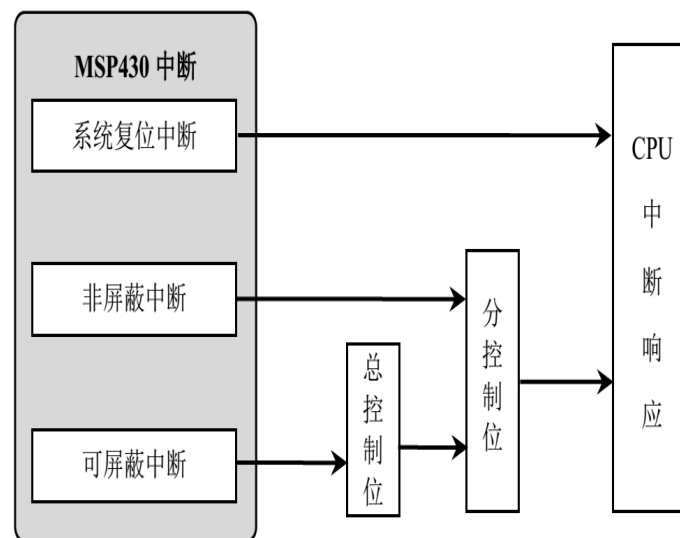
- 可见，非屏蔽中断也是多源中断。

4.1.2 中断类型



3. 可屏蔽中断

- 可屏蔽中断是由具备中断能力的片上外设(如定时器、DAC等)触发的中断。
- 可屏蔽中断能否被响应同时受控于总中断使能和各自模块的中断使能位。



4.1.2 中断类型



3. 可屏蔽中断

- 大部分片上外设的中断使能位在片上外设模块的寄存器中。
- 但是，也有一些片上外设(例如FLASH)的中断使能位在中断使能控制寄存器(IE1和IE2)中。

例如，看门狗定时中断使能位(WDTIE)在IE1中，部分串口收发中断使能位(UCB0TXIE、UCB0RXIE、UCA0TXIE、UCA0RXIE)在IE2中，系统复位后寄存器IE1和IE2将清零。

4.1 中断系统



- 4.1.1 中断系统基本概念
- 4.1.2 中断类型
- 4.1.3 中断响应过程
- 4.1.4 中断函数
- 4.1.5 鲲鹏920的中断系统

4.1.3 中断响应过程



- 中断处理是MSP430单片机的特色，也是**实现低功耗和提高CPU利用率的重要途径。**
- 理解整个中断响应过程将**有助于灵活应用MSP430单片机的众多中断。**

4.1.3 中断响应过程

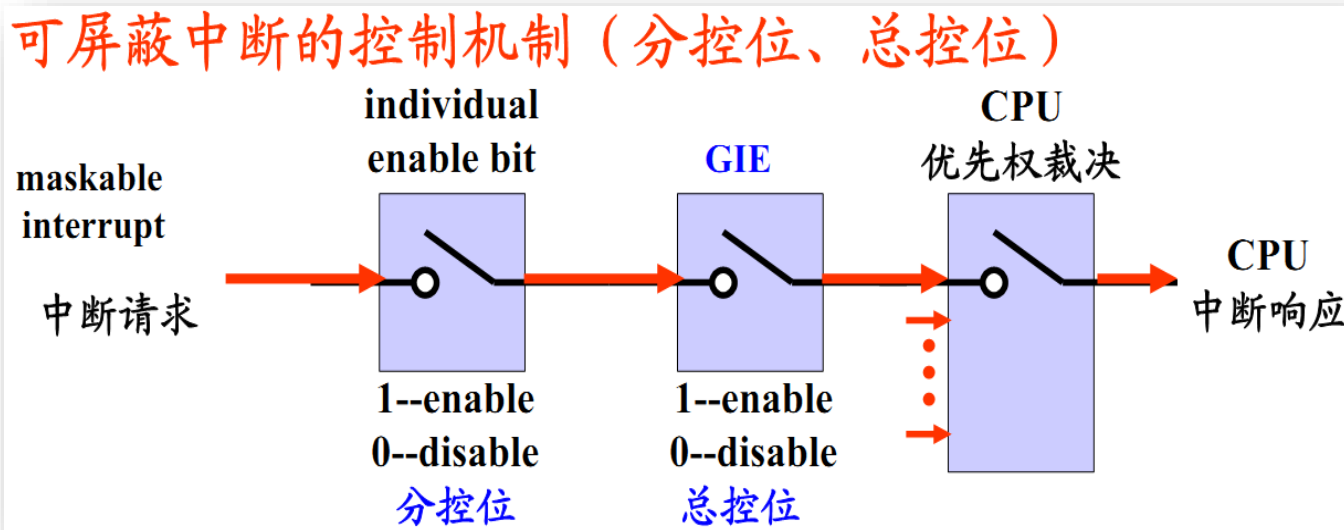


1. 中断通道设置

可屏蔽中断的中断通道设置主要包括

清除中断标志位、开启模块自身中断使能位、开启总中断使能位。

只要将这三方面做好了，中断请求信号就可以被畅通无阻地传送到中断仲裁机构进行裁决，以最终确定是否立即响应中断源的中断请求。



4.1.3 中断响应过程



2. 中断响应

- 中断响应是指MSP430单片机**对中断请求作出响应的过程**，也是从当前程序向中断服务程序过渡的过程。
- 若有多个优先级不同的中断源同时向CPU发出中断请求，CPU将根据优先级先响应优先级最高的中断请求。
- 当CPU接收中断源的中断请求后，便对该中断请求进行响应。

4.1.3 中断响应过程



2. 中断响应

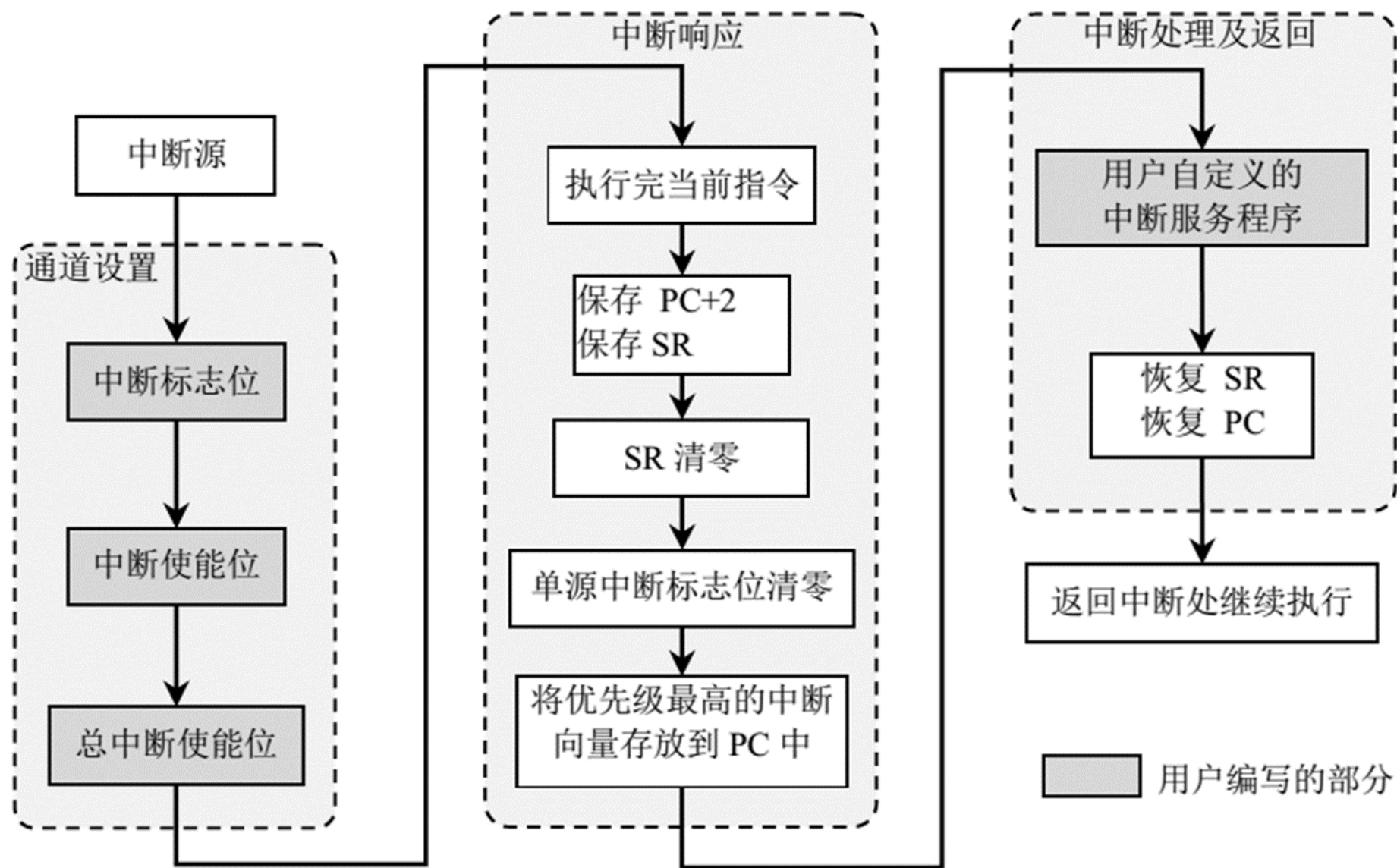
- CPU在执行完当前指令后，MSP430单片机硬件将自动执行以下中断响应过程：
 - ◆ 1). 保护PC
 - ◆ 2). 保护SR
 - ◆ 3). 优先级裁决
 - ◆ 4). 中断请求标志位清零
 - ◆ 5). SR清零
 - ◆ 6). 确定中断向量
 - ◆ 7). 执行中断服务程序

4.1.3 中断响应过程



3. 中断返回

- 当中断服务程序执行完后，就需要返回到主程序断点处继续执行下面的程序，这个过程就是**中断返回**。整个返回过程可分为下面三个步骤完成。
 - ◆ 1). 恢复SR
 - ◆ 2). 恢复PC
 - ◆ 3). 从断点处继续运行被中断的程序



中断处理全过程

4.1.3 中断响应过程



4. 中断时间

- 在中断处理的过程中有两个时间概念即**中断响应时间**与**中断返回时间**。
- 中断的**响应时间**就是中断的响应过程的时间，一般指从接受中断请求至执行中断服务程序前的时间。
- 中断**返回时间**是指从执行返回指令至执行主程序前的时间。

4.1.3 中断响应过程



4. 中断时间

- 对于MSP430单片机来说，中断响应时间为6个时钟周期，中断返回时间为5个时钟周期。
- 看门狗与外部复位中断引起的系统复位需要4个时钟周期。

注意事项



- 若有多个中断同时请求，CPU先响应优先级最高的中断请求。
- 当中断发生时，MSP430内部对应有一个标志被置1(置位)。中断程序之后，应确保该标志的值已清零，否则被当成又一次的中断申请。
- 对于单一中断标志的中断请求，CPU会自动清零该中断标志。
- 对于有多个中断标志的中断源请求，用户在中断子程序中用这些标志判断产生的具体子中断源，中断标志的清零由用户在使用完后编程清零。

4.1.3 中断响应过程



Example of an NMI Interrupt Handler

The NMI interrupt is a multiple-source interrupt. An NMI interrupt automatically resets the NMIIIE, OFIE and ACCVIE interrupt-enable bits. The user NMI service routine resets the interrupt flags and re-enables the interrupt-enable bits according to the application needs.

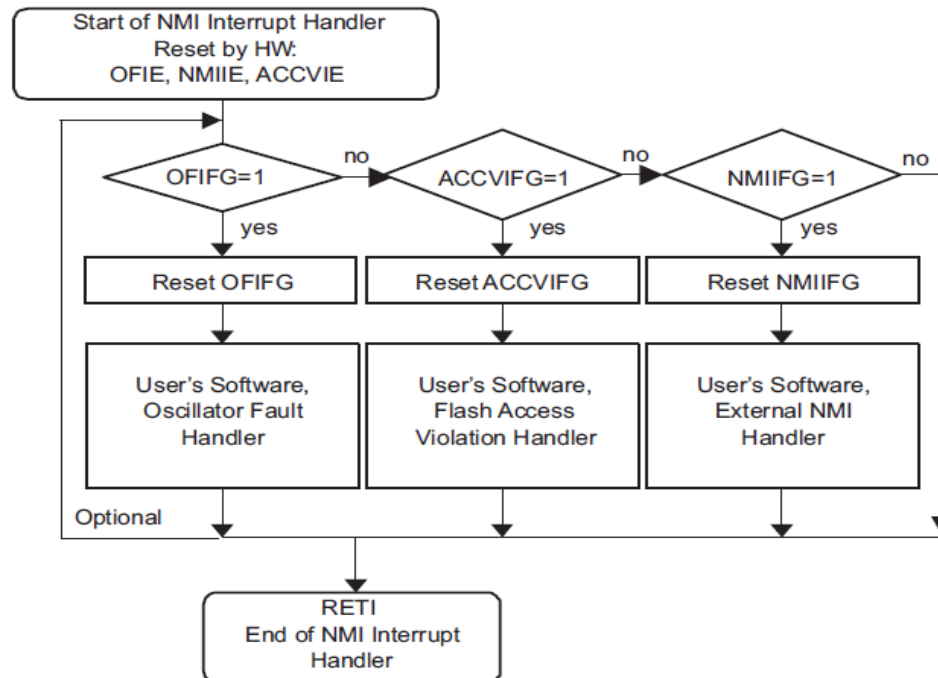


Figure 2-5. NMI Interrupt Handler

NOTE: Enabling NMI Interrupts with ACCVIE, NMIIIE, and OFIE

To prevent nested NMI interrupts, the ACCVIE, NMIIIE, and OFIE enable bits should not be set inside of an NMI interrupt service routine.

4.1 中断系统



- 4.1.1 中断系统基本概念
- 4.1.2 中断类型
- 4.1.3 中断响应过程
- 4.1.4 中断函数
- 4.1.5 鲲鹏920的中断系统

4.1.4 中断函数



- 中断函数是单片机 C 语言中的一个重要概念，它与汇编语言中的中断服务程序相对应。
- 与普通的自定义函数相比，中断函数是一个特殊的函数，它既没有任何输入参数，也没有函数返回值。

4.1.4 中断函数



- 在 MSP430 单片机 C 语言程序设计中，中断函数具有固定的定义格式，如下：

```
#pragma vector = 中断向量           // 指定中断入口地址(即中断向量)
__interrupt void 自定义函数名(void) // 为中断向量定义一个中断函数
{
    ..... // 中断服务程序的主体
}
```

- 单源中断源的中断标志寄存器会自动清零，但共源中断的中断标志寄存器需要手工清零。

4.1.4 中断函数



```
#pragma vector = PORT1_VECTOR
__interrupt void PORT1_ISR(void)
{
    //中断后想干的事写这里;
}
```

MSP430的中断子函数模板，加粗斜体部分是允许用户“创意”的地方，其他则是固定格式。

其中，PORT1_VECTOR是中断向量表中的地址宏定义，直接决定了这个中断子函数是响应哪个中断源。

PORT1_ISR是中断子函数名，这个最好按标准写法取名字，便于理解。中断向量表和标准的中断子函数名可见头文件<MSP430G2553.h>。



作业 2

列表总结MSP430G2553的中断向量，以及与中断源的对应关系。样例如下：

中断源	中断标志	中断类型	入口地址	中断向量
上电 外部复位 看门狗复位或安全键值出错 Flash安全键值出错	PORIFG RSTIFG WDTIFG	不可屏蔽 (系统复位)	0xFFFFE	RESET_VECTOR
NMI 振荡器失效 Flash访问出错	NMIIFG OFIFG ACCVIFG	非屏蔽	0xFFFFC	NMI_VECTOR



视频

- 7 鲲鹏920处理器的中断系统
- 8 鲲鹏920处理器中断执行流程

4.1 中断系统



- 4.1.1 中断系统基本概念
- 4.1.2 中断类型
- 4.1.3 中断响应过程
- 4.1.4 中断函数
- 4.1.5 鲲鹏920的中断系统



4.1.5 鲲鹏920的中断系统

通用中断控制器

GIC(General Interrupt Controller)

- SOC中进行中断管理和控制的部件



GIC版本

- GIC v2: ARM Cortex A7/A15/A53/A57

- GIC v3: ARM Cortex A53/A57/A72

nFIQ: 快速中断请求



4.1.5 鲲鹏920的中断系统

中断的分类

PPI (Private Peripheral Interrupt)

- 私有外设中断。PPI由核内硬件触发，一定是本核触发，本核处理。例如核内通用定时器的中断请求。

SPI (Shared Peripheral Interrupt)

- 共享外设中断。SPI是常见的外部设备中断，由外设触发，可以多核处理，不限定CPU。例如键盘按键，鼠标点击等。



4.1.5 鲲鹏920的中断系统

中断的分类

SIG (Software Generated Interrupt)

- 软件中断。SIG是通过软件写通用中断控制器中的SIG寄存器产生中断，通常用于核间通信。

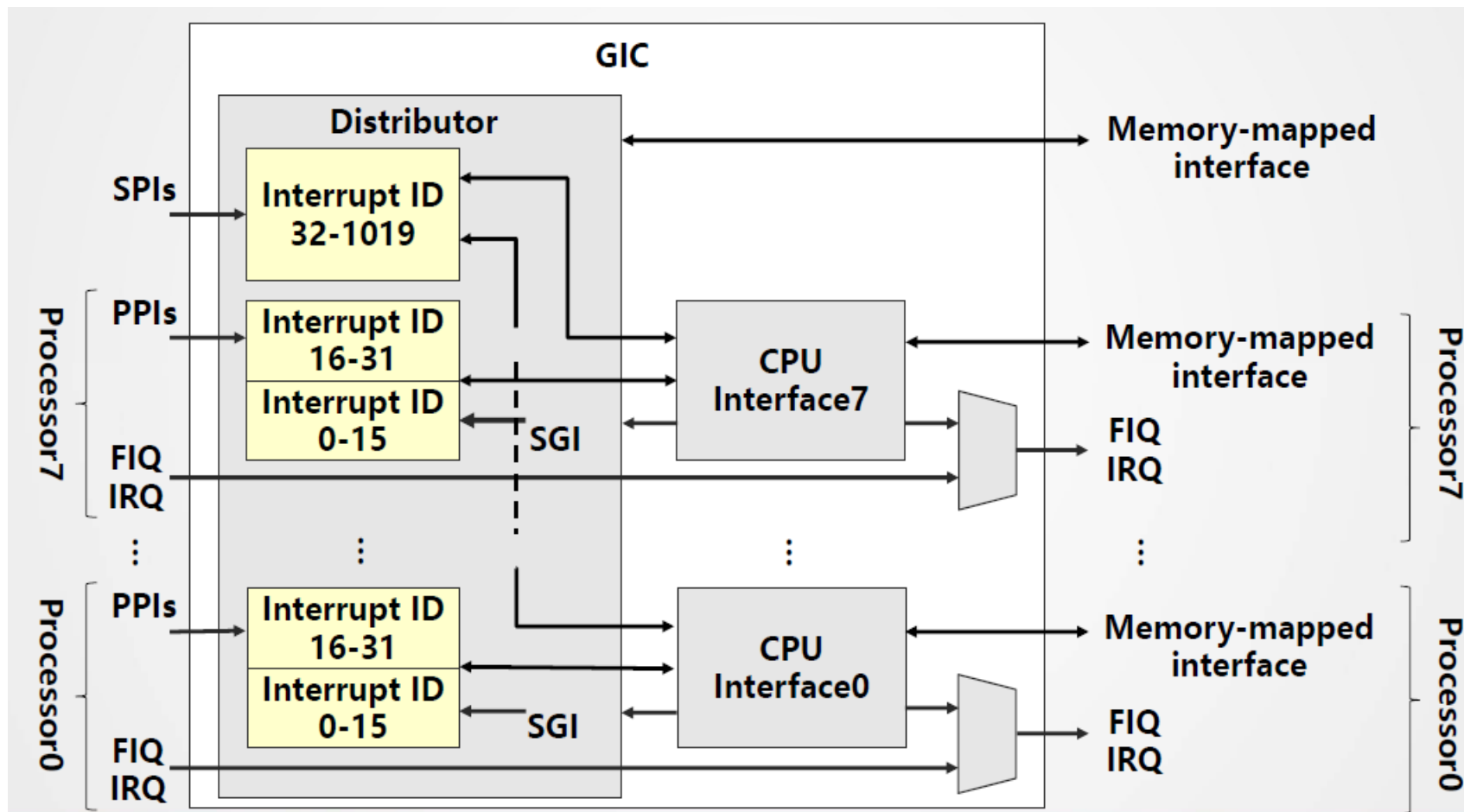
LPI (Locality-specific Peripheral Interrupt)

- 特定区域外设中断。LPI是边沿触发基于消息的中断，为支持消息中断 (MPI) 而设定。



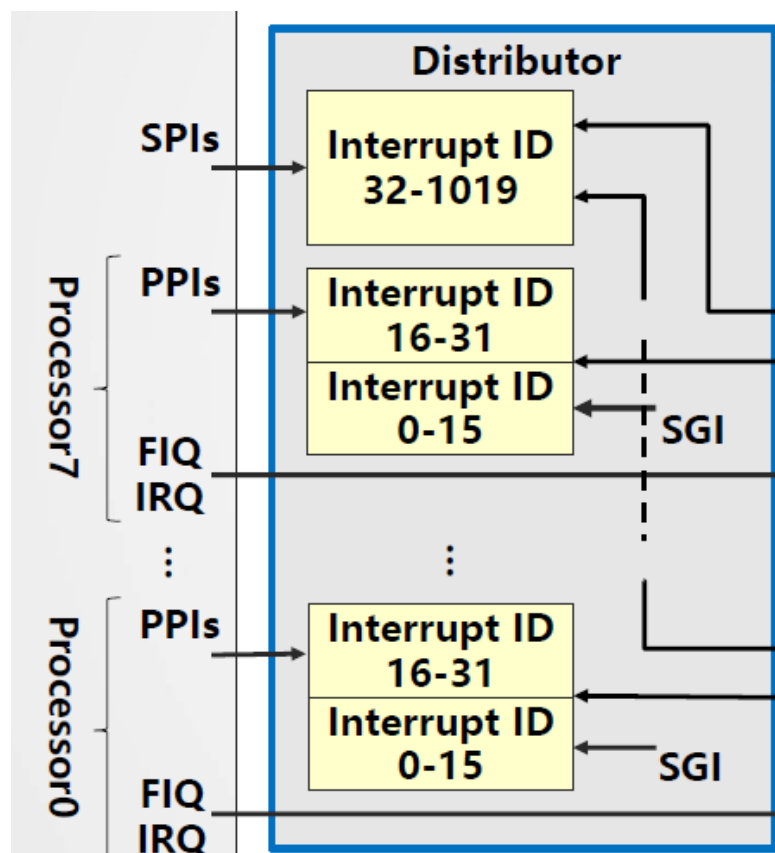
4.1.5 鲲鹏920的中断系统

GIC v2





4.1.5 鲲鹏920的中断系统



中断分发器功能:

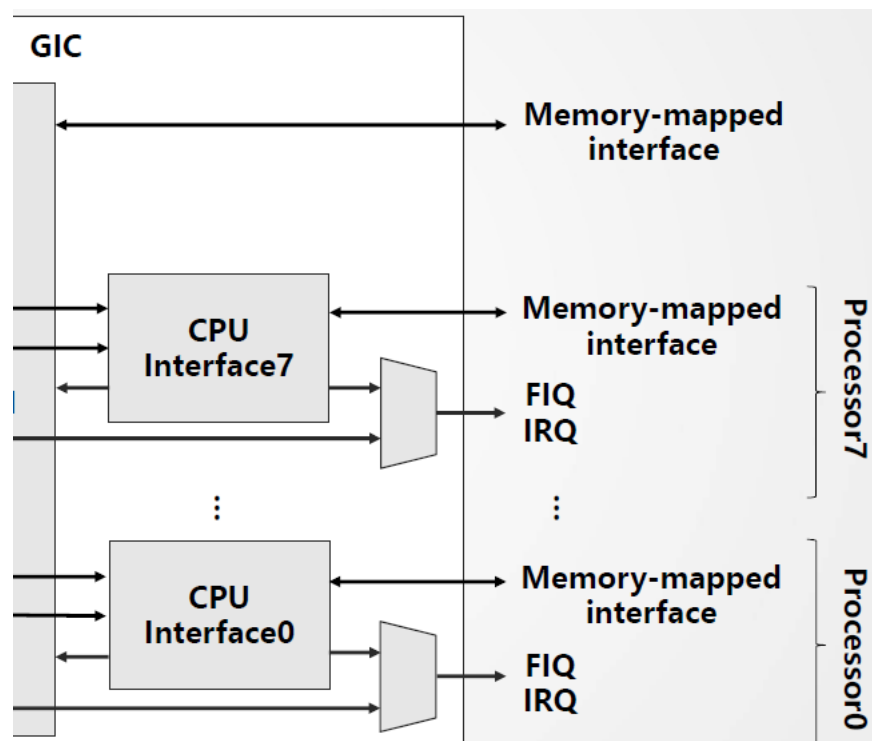
- 对整个中断控制器设备的使能等操作。
- 对每一个中断的优先级控制。
- 对每一个中断的触发方式设置。
- 决定将中断分发到哪个具体的CPU上进行处理。
- 记录每个中断的状态，是否到来，是否处理中，是否处理完，是否在等待发送状态等等。



4.1.5 鲲鹏920的中断系统

CPU接口功能:

- 使能和发送一个具体的中断信号到特定对应的CPU上，确认具体中断已经被CPU接受、处理，以及处理完成。
- 设置CPU能接受的中断的优先级别，以及对应的基于级别的中断抢占等处理。



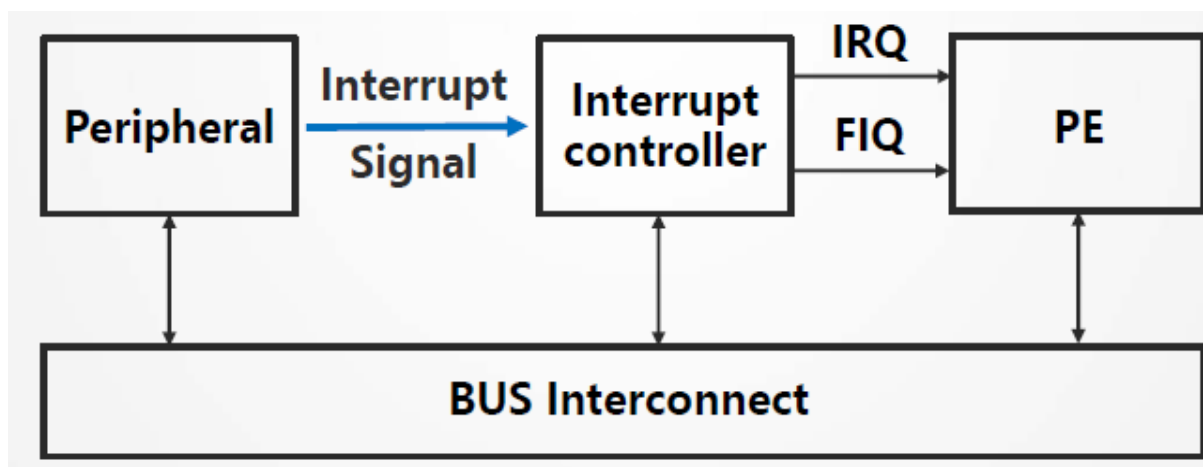


4.1.5 鲲鹏920的中断系统

对GIC v2的改进

➤传统中断信号线方式

利用独立的中断请求信号线向中断控制器发送中断请求。
中断信号线数目有限、不利于芯片布线、中断号数量不足。

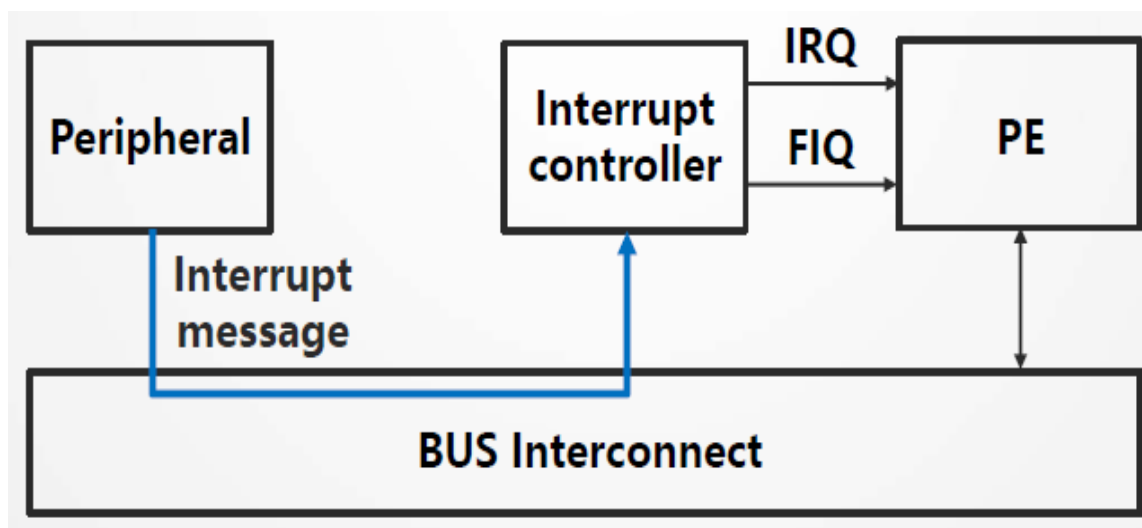




4.1.5 鲲鹏920的中断系统

对GIC v2的改进

- GIC v3新增了消息形式上报中断的机制（MBI: Message based Interrupt）：设备通过Bus Connection向特定地址空间写特定数据值来产生特定中断。

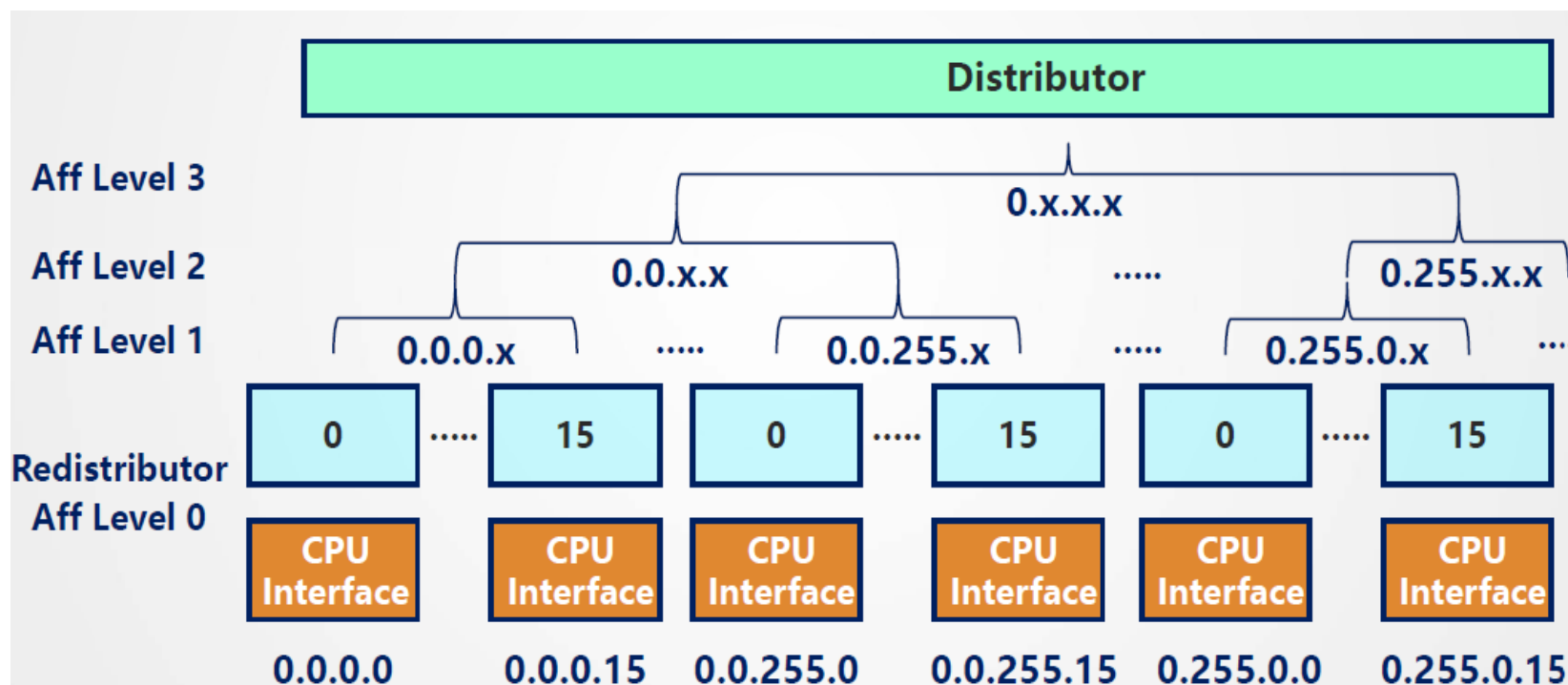




4.1.5 鲲鹏920的中断系统

对GIC v2的改进

- 改进内核编址方式，使用属性层次，以支持更多内核。





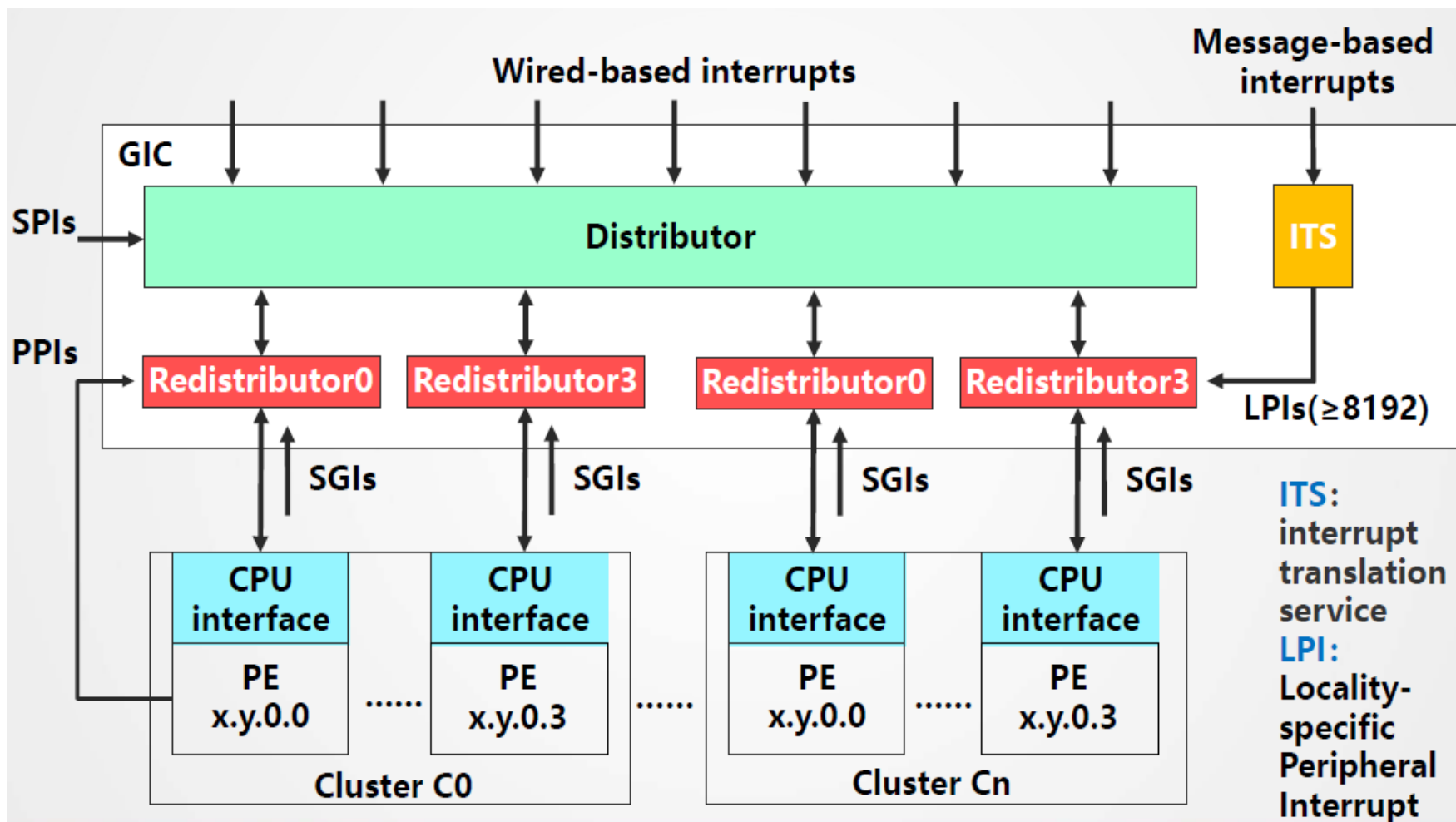
4.1.5 鲲鹏920的中断系统

对GIC v2的改进

- 由于CPU Interface的功能应用非常频繁，因此将其从GIC剥离出来，移入内核，以加快运行速度。
- 新增Redistributor模块
 - 使能和禁用SGI、PPI，设置其优先级，控制其状态
 - 设置PPI的触发方式
 - 处理PPI，使其不再经过分发器
 - 处理LPI，使其不需经过分发器
 - 负责相连内核的电源管理



4.1.5 鲲鹏920的中断系统

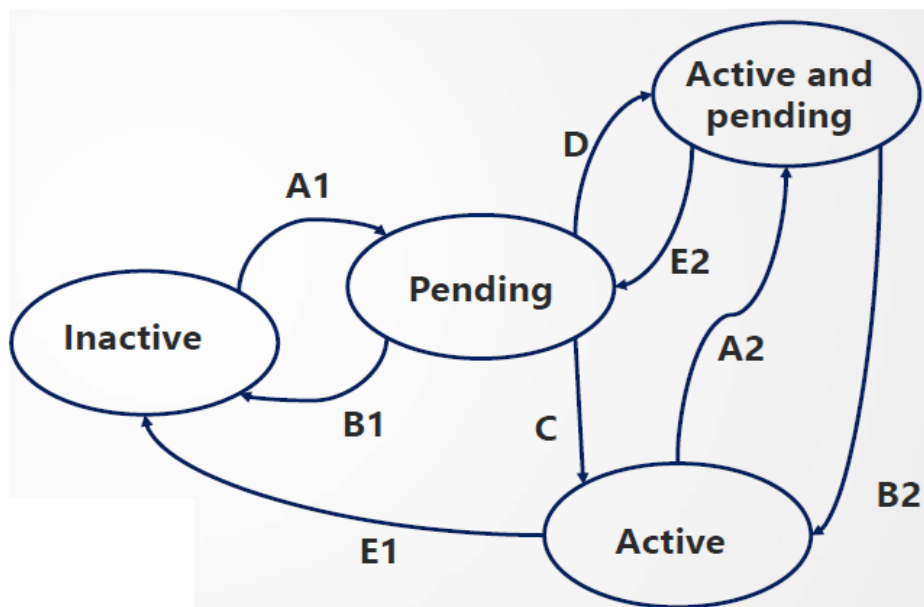




4.1.5 鲲鹏920的中断系统

中断状态

- inactive 中断处于无效状态。
- pending 中断处于有效状态，但CPU还没有响应该中断。
- active CPU正在响应该中断。
- active and pending CPU在响应该中断，但该中断源又有中断发送过来。

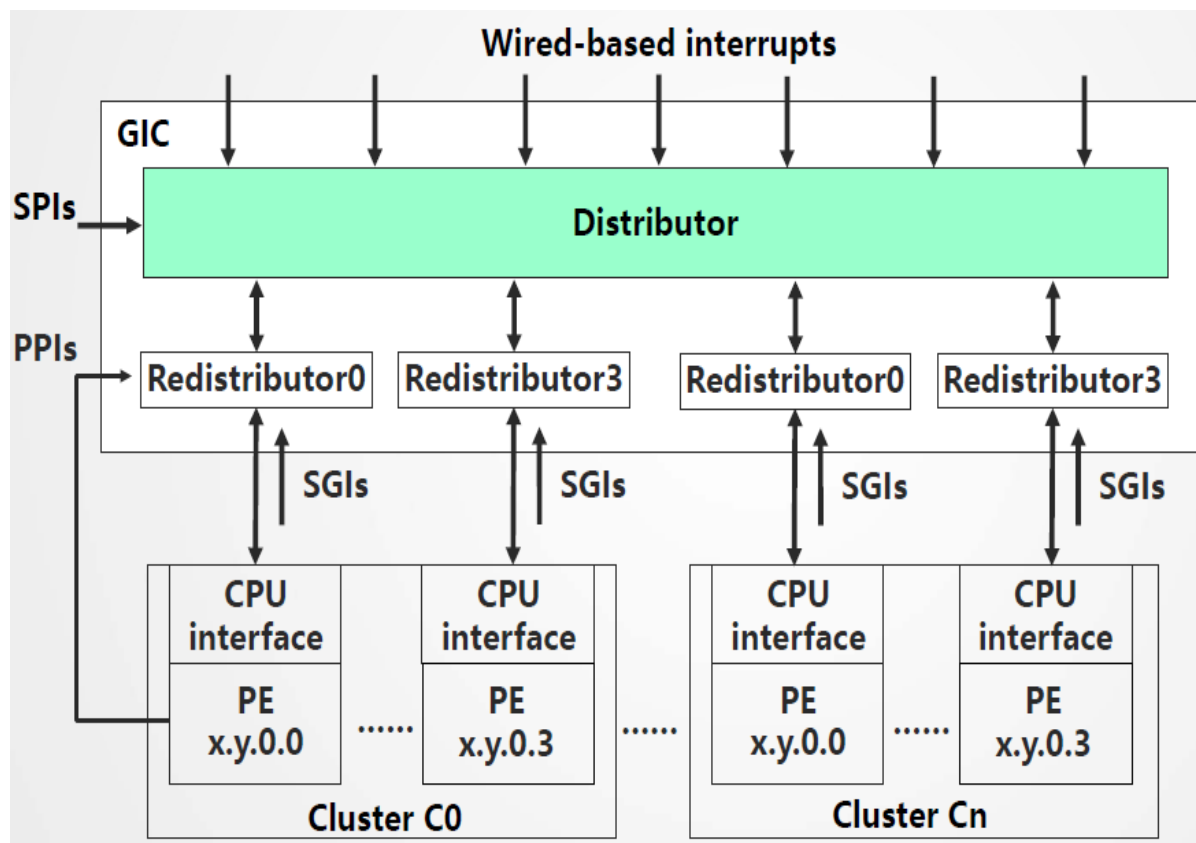




4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B为例



Step1:

Distributor工作

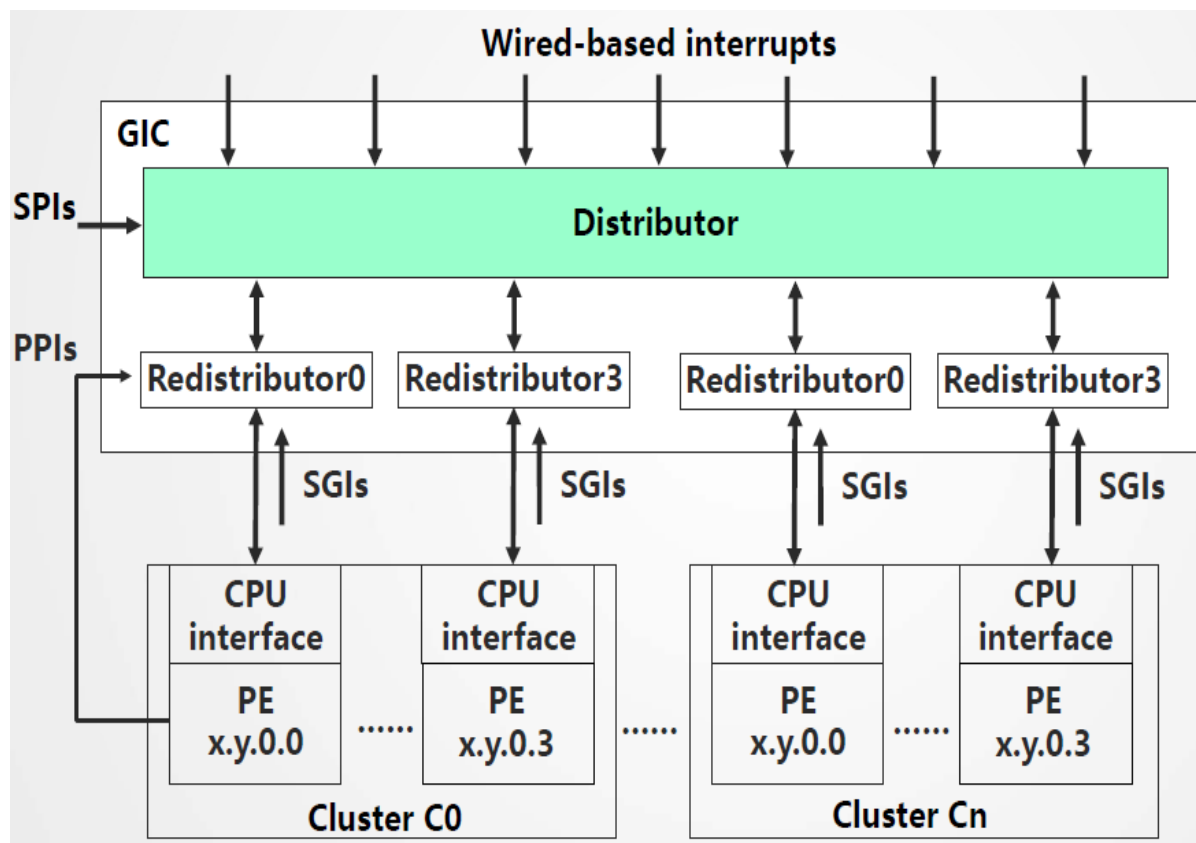
- 设置中断全局使能
- 设置按键中断使能
- 设置边沿触发



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step2:

Distributor工作

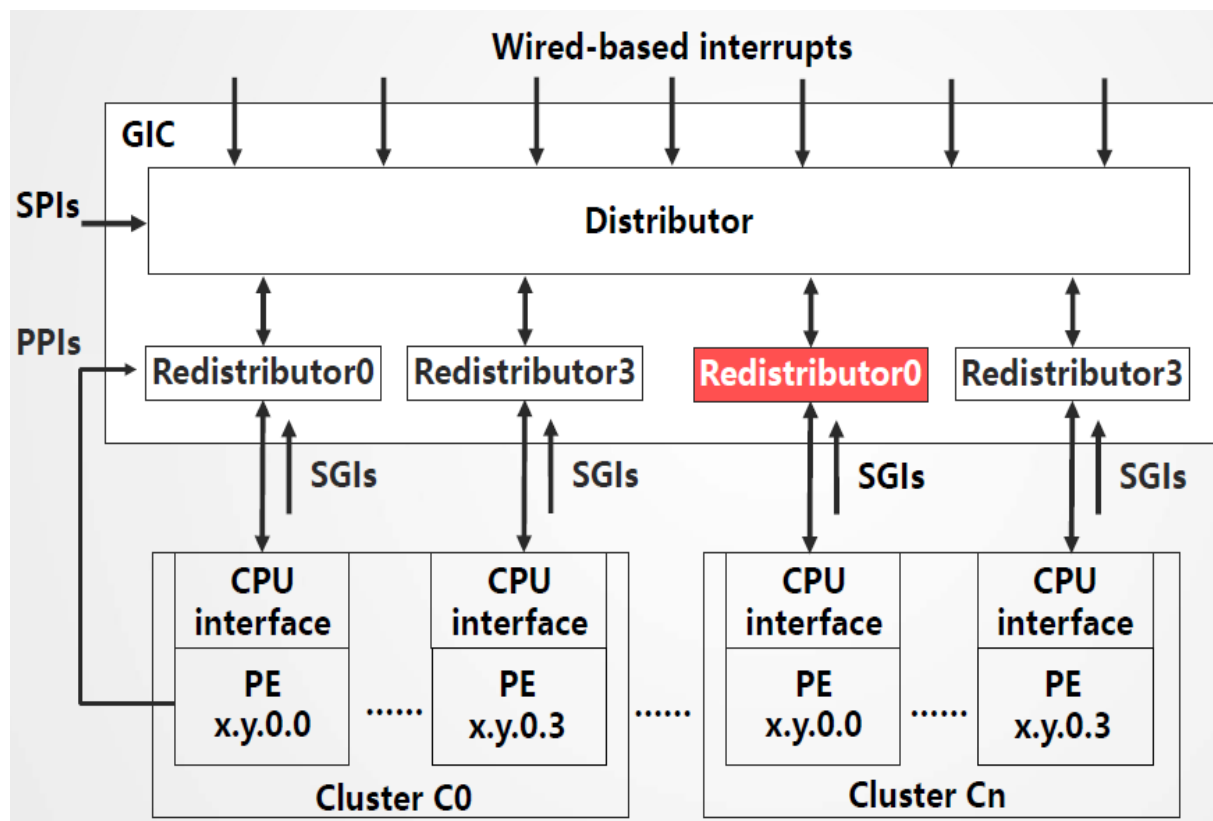
- 按键触发中断B
- 记录B的状态为 pending
- 判断B的优先级，如B优先级最高，则将B分给对应的 Redistributor



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step3:

Redistributor工作

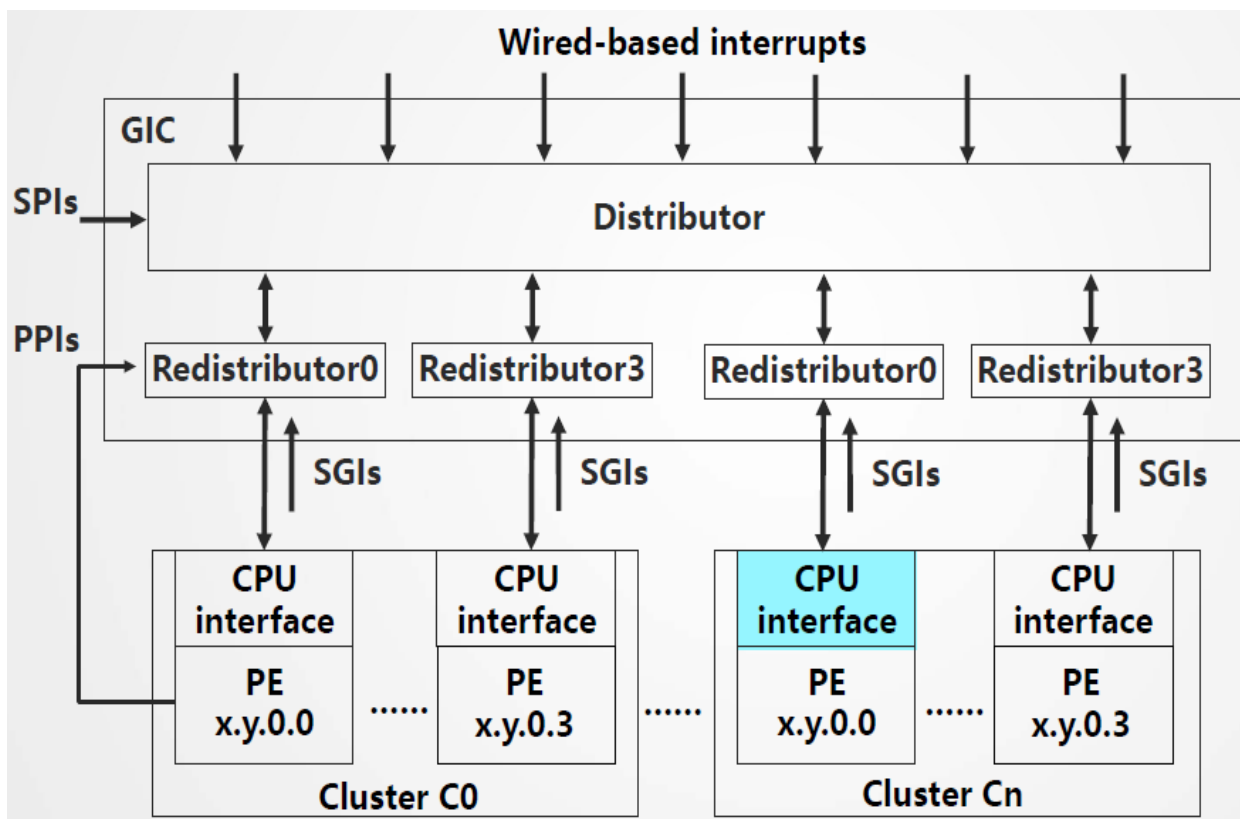
- 优先级排队（SGI、PPI、LPI）
- 如B最高，则送至CPU interface



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step4:

CPU interface工作

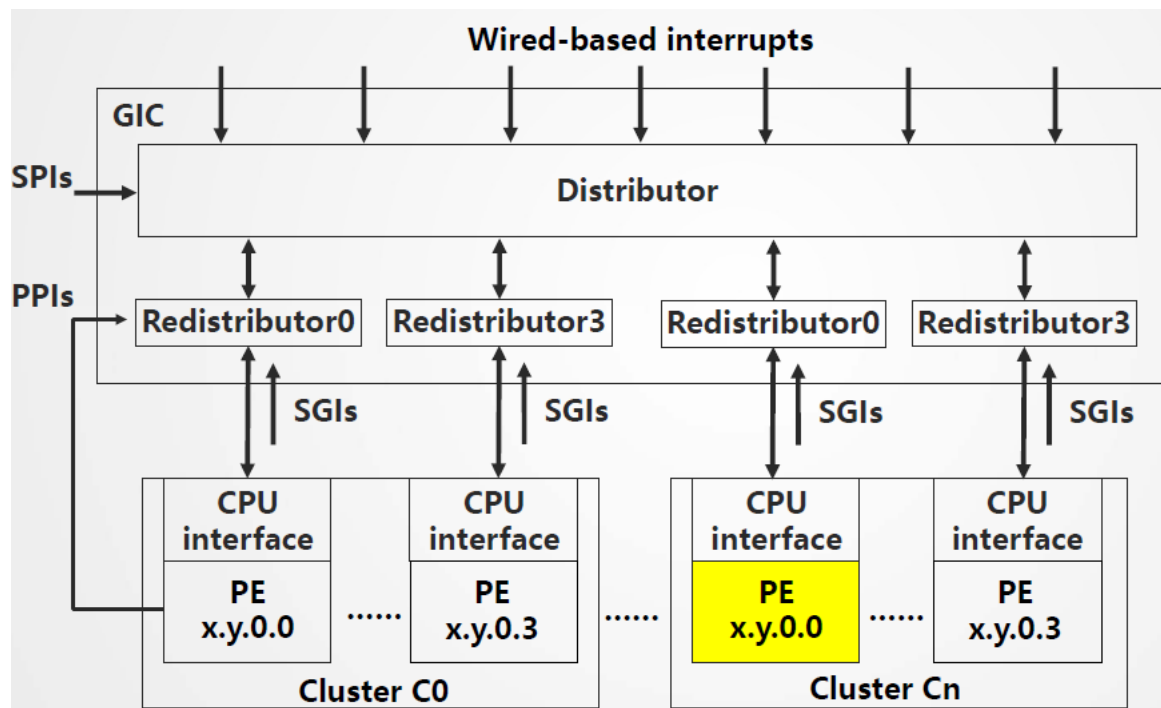
- 判断门槛优先级
- 如B未被屏蔽，且PE当前未执行级别更高的程序，则处理该中断。



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step5:

PE工作并设置状态

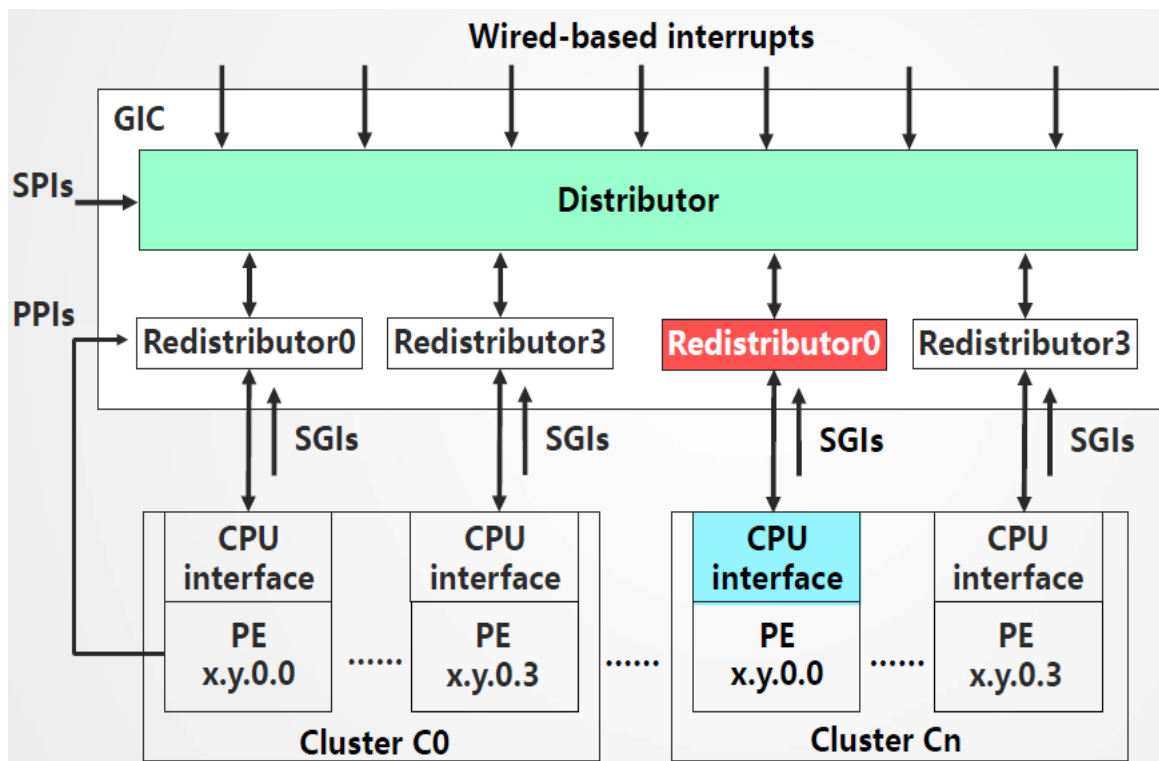
- 读取B的中断号。
- 同步通过硬件方式修改中断状态。



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step5:

修改中断状态

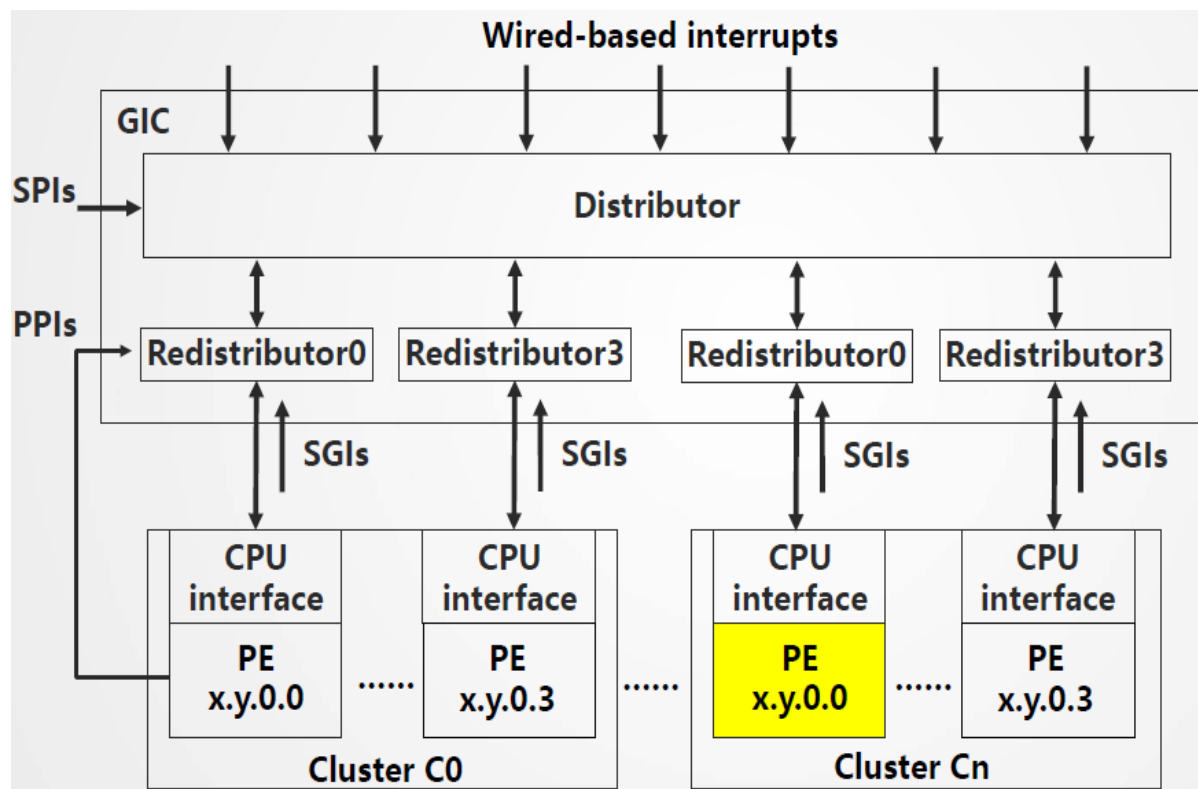
- CPU interface通知 Redistributor PE已处理中断
- Redistributor通知Distributor
- Distributor改中断状态为 Active
- 如此时再次按键，则状态改为Active and pending



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step6: PE工作

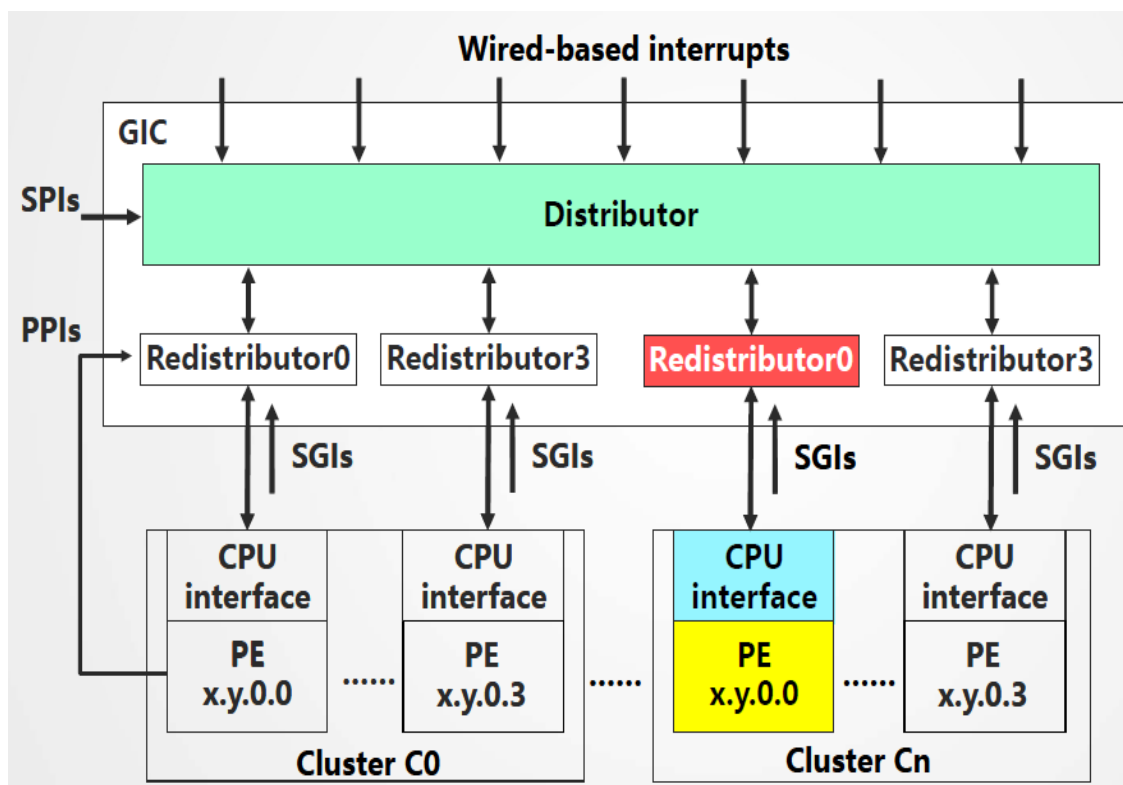
- 执行中断服务程序



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step7:

中断处理完成

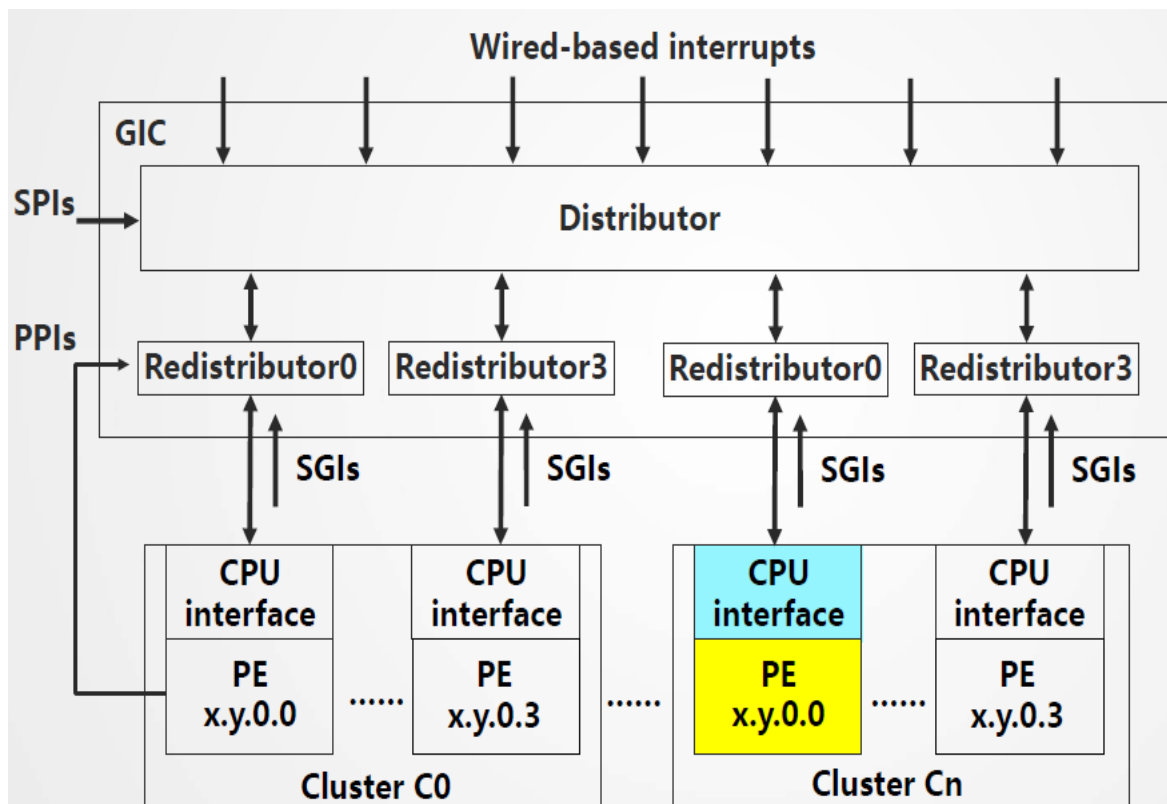
- PE通知CPU interface中断完成。
- interface通知Redistributor
- Redistributor通知Distributor
- Distributor改中断状态为Inactive



4.1.5 鲲鹏920的中断系统

SPI中断流程

按键中断B



Step*: 中断抢占

- PE正在处理中断B时，如有更高优先级的中断A来到，对应CPU interface，则发生抢占，PE转而执行中断A的服务程序。